2026 청년도약 인재양성 부트캠프

★ 기업 현장에서 배우며, 실무형 인재로 성장 ★

★ 실전인재형 ★ 점프인재형 ★ 실전인재형 ★ 점프인재형 ★ 실전인재형

인공지능 부트캠프

#지능형 프롬프트 엔지니어링
- 소프트웨어 설계

컴퓨터공학과

김 원 일

초보 프로젝트에서 자주 발생하는 실패

- “좋은 앱을 만들겠다”처럼 문제 정의가 너무 추상적
- 사용자가 누구인지 정하지 않고 기능을 나열한다.
- 필수 기능과 있으면 좋은 기능을 구분하지 않는다.
- AI에게 “만들어줘”라고만 지시해서 결과가 매번 달라진다.
- 완료 기준이 없어 테스트와 발표에서 설명이 어려워진다.
- 팀원별 역할과 담당 기능이 요구사항과 연결되지 않는다.

일반 Chat AI와 Coding Agent의 차이

- 결과를 '말로 설명 ' 하느냐, 프로젝트에 '직접 반영 ' 하느냐

일반 Chat AI

- 대화창 중심
- 코드 예시 제공
- 파일 직접 수정 불가
- 사용자가 복사·붙여넣기 필요
- 학습·아이디어 정리에 유용

AI Coding Agent

- 프로젝트 폴더 중심
- 파일 생성·수정 가능
- 명령 실행과 테스트 가능
- 변경사항 확인 필요
- 실제 개발 작업에 유용

Agent 사용 전략

- 처음 부터 여러 도구를 섞지 않고 역할을 나눔
  - Agent의 특징에 따라 적용할 단계를 개발자가 지정하고 적용
  - 개발자의 역량과 개발 방법에 따라 달라질 수 있음
  - 단순 예제이므로 참고용으로만 사용하고, 본인만의 원칙을 세울 것

1단계

Claude Code 중심
프로젝트 폴더에서 실행
파일 생성·수정 실습

2단계

Codex IDE/CLI 보조
코드 리뷰·설명·수정 제안
수업 후반부터 적극 활용

공통 원칙

Git 체크포인트
수정 범위 제한
변경사항 검토 후 적용

Agent 사용 시 권한과 보안

- 편리함보다 안전한 개발 습관이 먼저
  - 주요 소프트웨어 및 접근 권한 정보는 코드에 직접 기록하지 않음
  - 개발 단계를 본인이 지정하고, 단계마다 소스 코드 형상 관리를 수행

주의할 점

API Key를 코드에 직접 쓰지 않는다.
Agent가 제안한 명령을 읽고 실행한다.
프로젝트 외부 파일 수정 권한을 주의한다.
개인정보·비밀번호를 프롬프트에 넣지 않는다.

안전한 습관

작업 전 git status 확인
작업 전후 commit 생성
변경 diff 확인
모르는 명령은 먼저 설명 요청

Agent에게 좋은 지시를 내리는 형식

- 수정 범위와 확인 기준 동시에 제시
  - 코드를 포함한 다양한 수정에서 기본 형식을 항상 지키도록 제시
  - 정확한 지시를 통해 원하는 형태로의 변경을 요청

기본 형식

현재 상황: 어떤 파일/기능을 다루는가
작업 요청: 무엇을 바꿀 것인가
제약 조건: 건드리면 안 되는 부분
검증 기준: 실행/테스트 방법

예시

score.py에서 입력 검증만 추가해줘.
기존 평균 계산 로직은 바꾸지 마.
숫자가 아닌 값이 들어오면 다시 입력받게 해줘.
수정 후 실행 방법을 알려줘.

인공지능을 활용한 표준 개발 흐름

- 개발 단계별 설정
  - 고정되어 있는 형식이 아니므로 본인에게 맞는 개발 흐름을 찾을 것
  - 각 단계마다 다른 인공지능을 사용하기 때문에 반드시 문서화할 것
  - 문서화된 내용을 다음 단계에 전달해야 이전 단계의 내용이 유지됨
    - 지속적인 인공지능 교체가 이루어진다면 단계별 문서화는 필수
    - 특정 단계 내용 변경이 필요하다면 이전 정보를 먼저 전달하고, 변경 내용을 생성하도록 지시
    - 테스트 단계에서 요구사항의 문제가 확인되면, 요구사항 점검 후 적용할 때도 유용
  - 단계 별 정보 보관과 유지가 개발자에게 필요함
    - 개발자가 직접 수행하지 않아도 지시는 개발자가 직접해야 함
  - 설계 전에 가장 중요한 단계는 요구사항 단계



1단계: 문제 정의

왜 문제 정의부터 하는가

- 문제 정의는 프로젝트의 방향을 정하는 첫 단계이다.
- 사용자가 겪는 불편함과 해결하고 싶은 상황을 짧고 명확하게 정리한다.
- 좋은 문제 정의는 기능 목록보다 먼저 작성되어야 한다.

문제 정의 템플릿

- 사용자는 누구인가?
- 무엇이 불편한가?
- 이 프로그램은 무엇을 해결하는가?
- 성공 기준은 무엇인가?

문제 정의가 모호하면 AI는 임의로 기능을 확장하거나 다른 방향으로 구현할 수 있다.

문제 정의 예시

나쁜 예

할 일 관리 프로그램을 만든다.

- 사용자와 문제 상황이 불분명하다.
- 어떤 기능이 필요한지 판단하기 어렵다.

좋은 예

개인 학습자가 터미널에서 간단히 오늘 할 일을 기록하고 완료 여부를 확인할 수 있는 프로그램을 만든다.

- 복잡한 회원가입 없이 파일에 데이터를 저장한다.
- 목표는 빠르게 실행 가능한 CLI 도구를 만드는 것이다.

좋은 문제 정의는 사용자·문제·해결방향을 모두 포함한다.

2단계: 요구사항 작성

기능 요구사항(FR)

- 시스템이 반드시 수행해야 하는 기능
- 예: 할 일 추가, 목록 조회, 완료 처리, 삭제
- 사용자 행동과 직접 연결됨

비기능 요구사항(NFR)

- 성능, 사용성, 보안, 유지보수성 등 품질 조건
- 예: 터미널에서 쉽게 사용할 수 있어야 함
- 데이터 파일이 손상되지 않아야 함

요구사항은 기능과 품질 조건을 구분해서 정리한다.

왜 요구사항 분석이 중요한가?

요구사항이 약할 때 생기는 문제

- 기능은 많지만 사용자가 원하는 것이 아님
- AI가 임의로 기능을 추가하거나 구조를 바꿈
- 팀원마다 서로 다른 결과물을 상상함
- 테스트 기준이 없어 완성 여부를 판단하기 어려움

요구사항이 좋을 때 생기는 효과

- 프로젝트 목표가 명확해짐
- AI에게 더 정확한 지시가 가능함
- 설계와 구현의 기준이 생김
- 중간·최종 발표에서 설명력이 높아짐

MiniTask 요구사항 예시

기능 요구사항

- FR-01 사용자는 할 일을 추가할 수 있다.
- FR-02 사용자는 전체 할 일 목록을 조회할 수 있다.
- FR-03 사용자는 특정 할 일을 완료 상태로 변경할 수 있다.
- FR-04 사용자는 특정 할 일을 삭제할 수 있다.

비기능 요구사항

- NFR-01 데이터는 JSON 파일에 저장한다.
- NFR-02 명령어 오류 시 사용법을 안내한다.

AI에게 전달하기 전 요구사항을 표준 형식으로 정리한다.

3단계: 설계하기

설계란 무엇인가

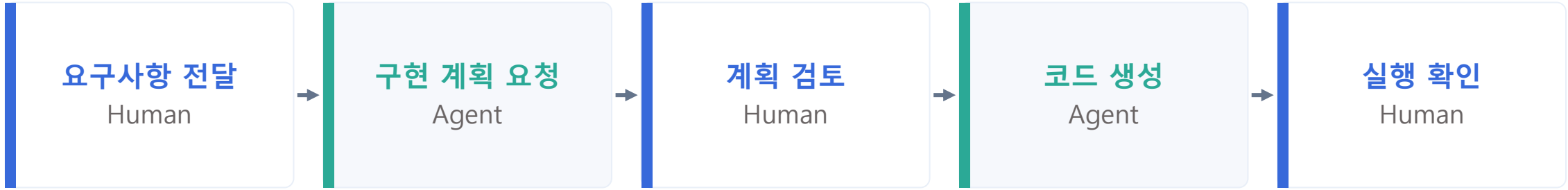
- 설계는 요구사항을 코드 구조로 바꾸는 과정이다.
- 어떤 파일과 모듈이 필요한지, 각 모듈의 역할을 정한다.
- AI에게 바로 코드를 만들라고 하기보다 먼저 구조를 설계하게 해야 한다.
- 설계가 있어야 코드 리뷰와 테스트도 쉬워진다.

핵심 모듈

```
main.py
task_service.py
storage.py
tasks.json
```

구현 전에 구조를 먼저 만든다 — 설계 없는 구현은 수정 비용을 키운다.

4단계: Agent에게 구현 지시하기



권장 방식: '바로 코딩하지 말고 먼저 구현 계획을 설명해줘'라고 지시한다.
계획이 맞으면 그때 파일 생성과 코드 구현을 요청한다.

바로 구현보다 '계획 → 확인 → 구현' 흐름이 안전하다.

Agent 지시문: 나쁜 예와 좋은 예

나쁜 지시

`todo 프로그램 만들어줘.`

- 범위가 불명확하다.
- 저장 방식, 파일 구조, 명령어 형식이 없다.
- AI가 임의로 구조를 만들 가능성이 높다.

좋은 지시

- Python CLI로 MiniTask를 구현해줘.
- 파일 구조는 src/main.py, src/task_service.py, src/storage.py로 분리해줘.
- 데이터는 data/tasks.json에 저장해줘.
- 먼저 구현 계획을 설명하고, 내가 확인한 뒤 코드를 작성해줘.

구현 범위와 파일 구조를 명확히 지정한다.

5단계: 테스트하기

테스트 원칙

- 정상 입력만 확인하면 충분하지 않다.
- 없는 명령어, 잘못된 ID, 빈 입력, 파일 없음 상황도 확인해야 한다.
- 테스트 케이스는 요구사항과 직접 연결되어야 한다.
- Codex 또는 Agent에게 테스트 케이스 생성을 요청할 수 있다.

테스트 예시

- 할 일 추가 후 목록에 보이는가?
- 완료 처리 후 상태가 바뀌는가?
- 없는 ID를 처리할 때 오류가 나는가?
- JSON 파일이 없을 때 자동 생성되는가?

테스트는 AI가 만든 코드를 믿기 위한 최소 조건이다.

사람이 반드시 확인해야 할 체크리스트

결과 확인

- 요구사항을 모두 충족하는가?
- 요구하지 않은 기능이 추가되지 않았는가?
- 실행 방법이 명확한가?
- 오류 상황을 처리하는가?

구조와 기록 확인

- 파일 구조가 설계와 일치하는가?
- 코드를 설명할 수 있는가?
- 테스트 결과가 남아 있는가?

AI가 만들었다고 끝난 것이 아니다.

요구사항 분석의 기본 흐름



핵심 원칙

요구사항 분석은 "무엇을 만들까?"가 아니라 "누구의 어떤 문제를 어떤 기준으로 해결할까?"를 정리하는 과정이다.

1단계: 문제 정의

문제 정의에 들어갈 내용

- 현재 어떤 불편함이 있는가?
- 누가 이 문제를 겪는가?
- 왜 이 문제가 중요한가?
- 기존 방식의 한계는 무엇인가?
- 우리가 해결하려는 범위는 어디까지인가?

GPT에게 요청하기

다음 프로젝트 아이디어를 문제 정의 관점에서 정리해줘.

아이디어: 회의 내용을 자동으로 정리하는 웹 서비스

반드시 포함할 것:
1. 문제 상황
2. 주요 사용자
3. 기존 방식의 한계
4. 해결 범위
5. 제외할 범위

문제 정의 템플릿

문제 정의 문장 구조

- [사용자]는 [상황]에서 [문제]를 겪고 있으며, 이는 [영향]을 발생시킨다.
  따라서 본 프로젝트는 [해결 방향]을 제공한다.

예시

- 대학생 팀 프로젝트 팀원은 회의 후 결정사항과 담당 업무를 잊는 문제를 겪고 있으며, 이는 과제 지연과 중복 작업을 발생시킨다.
  따라서 본 프로젝트는 회의 내용을 입력하면 핵심 요약과 Action Item을 자동 생성하는 도구를 제공한다.

2단계: 사용자와 이해관계자 분석

사용자(User)

- 실제로 서비스를 사용하는 사람
- 주요 기능을 직접 실행하는 사람
- 사용 편의성과 흐름이 중요함
- 예: 학생, 교수, 팀장, 관리 담당자

이해관계자(Stakeholder)

- 서비스 결과에 영향을 받는 사람
- 운영·관리·평가와 관련된 사람
- 정책·예산·보안 요구가 중요함
- 예: 관리자, 기관, 고객, 평가자

사용자 페르소나 작성

페르소나 항목

- 이름/역할
- 목표
- 현재 불편함
- 사용 환경
- 기대 기능
- 기술 숙련도

예시: 팀 프로젝트를 수행하는 대학생

- 역할: 조별과제 팀장
- 목표: 회의 결과를 빠르게 정리하고 담당자를 분명히 하고 싶음
- 불편함: 회의 후 정리 시간이 오래 걸리고 누가 무엇을 해야 하는지 잊음
- 기대 기능: 회의 요약, 담당자별 할 일, 다음 회의 안건 자동 정리

3단계: 기능 요구사항(FR)

기능 요구사항의 특징

- 사용자가 수행할 수 있는 동작으로 표현한다
- 시스템이 제공해야 하는 기능을 명확히 적는다
- 가능하면 식별번호를 붙인다
- 테스트 가능한 문장으로 작성한다

예시

- FR-001 사용자는 회의 제목과 내용을 입력할 수 있다.
- FR-002 시스템은 입력된 회의 내용을 저장한다.
- FR-003 시스템은 회의 내용을 요약한다.
- FR-004 시스템은 Action Item을 추출한다.
- FR-005 사용자는 저장된 회의 목록을 조회할 수 있다.

4단계: 비기능 요구사항(NFR)

비기능 요구사항 유형

- 성능: 응답시간, 처리량
- 보안: 인증, 권한, 데이터 보호
- 사용성: 화면 흐름, 오류 안내
- 신뢰성: 장애 대응, 데이터 보존
- 확장성: 사용자 증가 대응

예시

- NFR-001 요약 결과는 10초 이내에 표시되어야 한다.
- NFR-002 사용자의 API Key는 코드에 직접 저장하지 않는다.
- NFR-003 잘못된 입력에는 명확한 오류 메시지를 제공한다.
- NFR-004 주요 기능은 README에 실행 방법을 포함해야 한다.

기능 요구사항과 비기능 요구사항 비교

FR · 기능 요구사항

- 의미: 시스템이 제공해야 하는 기능
- 예시: 회의 내용을 저장한다
- 검증: 저장 버튼 클릭 후 목록에 표시되는지 확인

- 의미: 사용자가 할 수 있는 행위
- 예시: 사용자는 회의 목록을 조회한다
- 검증: 목록 API/화면 테스트

NFR · 비기능 요구사항

- 의미: 시스템의 품질 조건
- 예시: 요약은 10초 이내 표시한다
- 검증: 응답시간을 측정한다

- 의미: 보안·성능·사용성 조건
- 예시: API Key는 환경변수로 관리한다
- 검증: 코드와 설정 파일 점검

5단계: User Story 작성

User Story 기본 형식

- As a [사용자], I want [기능], so that [목적/가치].

예시 1

- As a 팀장, I want 회의 내용을 요약하고 싶다,
  so that 팀원에게 빠르게 공유할 수 있다.

예시 2

- As a 팀원, I want 내 담당 Action Item만 보고 싶다,
  so that 내가 해야 할 일을 빠르게 확인할 수 있다.

6단계: 수용 기준 작성

수용 기준이 필요한 이유

- 요구사항이 완료되었는지 판단할 수 있다
- 테스트 케이스의 기준이 된다
- 팀원과 AI가 같은 목표를 보게 한다
- 중간·최종 발표에서 검증 근거가 된다

예시: 회의 요약 기능

- 회의 내용을 입력하면 요약 버튼이 활성화된다.
- 요약 결과는 핵심 내용 3~5개로 표시된다.
- 빈 입력일 경우 오류 메시지를 표시한다.
- 요약 실패 시 재시도 안내를 제공한다.

7단계: 우선순위 정하기

Must Have

- 프로젝트의 핵심 기능
- 없으면 서비스가 성립하지 않음
- 중간 발표 이후 반드시 구현

Should Have

- 중요하지만 대체 가능
- 시간이 허용되면 구현
- 핵심 기능 이후 작업

Could Have

- 있으면 좋은 기능
- 발표용 확장 기능
- 최종 단계에서 선택 구현

요구사항 정의서 구조

1. 프로젝트 개요
2. 문제 정의
3. 주요 사용자 및 이해관계자
4. 기능 요구사항(FR)
5. 비기능 요구사항(NFR)
6. User Story
7. 우선순위
8. 제외 범위
9. 검토 결과 및 수정 사항

AI를 활용한 요구사항 분석 프롬프트

요구사항 분석 프롬프트

```
너는 소프트웨어 요구사항 분석가다.
다음 아이디어를 바탕으로 요구사항을 정리해줘.

아이디어: 팀 회의 내용을 입력하면 AI가 요약과 Action Item을 생성하는 웹 서비스

출력 형식:
1. 문제 정의    2. 주요 사용자    3. 기능 요구사항(FR-001 형식)
4. 비기능 요구사항(NFR-001 형식)    5. User Story 5개
6. 주요 기능 5개    7. 제외할 기능
```

좋은 요구사항 프롬프트의 조건

부족한 프롬프트

- “회의록 앱 요구사항 만들어줘.”
- 문제, 사용자, 범위가 불명확함
- 출력 형식이 없어 결과가 매번 달라짐
- 검토 기준이 없음

좋은 프롬프트

- 역할을 지정한다: 요구사항 분석가
- 아이디어와 사용자를 설명한다
- FR/NFR/User Story 형식을 요구한다
- 제외 범위까지 요청한다

설계 실습

파일에서 특정 문자열을 검색하는 프로그램을 작성하려고 해. 기존 프로그램에서는 한번 확인되면 동작을 멈췄는데, 이번에는 파일 전체에서 검색되는 모든 위치에 대해 리스트 형식으로 출력해서 더 많은 정보를 제공하려 하는데, 문제 정의와 요구사항을 문서로 만들어보자.

